

深圳大学考试答题纸

(以论文、报告等形式考核专用)
二〇二五~二〇二六 学年度第 二 学期

课程编号	1502850001	课程名称	互联网编程	主讲教师	Basker George	评分	
学号	2024150171	姓名	陈靖果	专业年级	计算机科学与技术(创新班)		
学号	2024150138	姓名	李培鑫	专业年级	计算机科学与技术(创新班)		
学号	2024150066	姓名	苏泽铭	专业年级	计算机科学与技术(创新班)		
学号	2024150126	姓名	何展明	专业年级	软件工程		

教师评语: The mark obtained for different categories as per the grading policy are as follows

序号	评分点 (分值)	成绩
1	Correct implementation of socket communication (TCP/UDP) (10 pts).	
2	Proper handling of multithreading/NIO (10 pts).	
3	Security features (SSL, password hashing) (10 pts).	
4	HTTP protocol compliance (if applicable) (10 pts).	
5	Code Structure (10 pts)	
6	User Experience (10 pts)	
7	Innovation (10 pts)	
8	Report (30)	
合计		100



Basker George
05 JULY 2026

Multi-Threaded Client-Server Chat Application
with Encryption

题目

Experimental Procedures

一、NIO-Based TCP Communication and Multi-Client Concurrent Processing

Clients establish TCP connections via `java.net.Socket` and conduct line-based text communication using `\n` as the message delimiter.

The server adopts the Java NIO architecture consisting of `Selector`, `ServerSocketChannel` and `SocketChannel`. A single `Selector` thread listens for the `OP_ACCEPT`, `OP_READ` and `OP_WRITE` events of all channels with non-blocking processing. Once messages arrive, `MessageCodec` decodes data from the `ByteBuffer` by splitting on `\n`, then dispatches the decoded messages asynchronously to a `ThreadPool` (thread count equal to the number of CPU cores) for routing, preventing blocking of the I/O thread. Write operations are buffered with a `ConcurrentLinkedQueue`; if data cannot be fully written in one attempt, the `OP_WRITE` event is registered to resume writing upon the next trigger.

Selector Event Loop & Event Dispatching

```

public void start() throws IOException {
    selector = Selector.open();
    serverChannel = ServerSocketChannel.open();
    serverChannel.configureBlocking(false);
    serverChannel.bind(new InetSocketAddress(PORT));
    serverChannel.register(selector, SelectionKey.OP_ACCEPT);

    System.out.println("ChatServer started on port " + PORT);

    while (running) {
        try {
            if (selector.select() == 0) continue;
            Set<SelectionKey> keys = selector.selectedKeys();
            Iterator<SelectionKey> it = keys.iterator();
            while (it.hasNext()) {
                SelectionKey key = it.next();
                it.remove();
                handleKey(key);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

private void handleKey(SelectionKey key) throws IOException {
    if (!key.isValid()) return;

    if (key.isAcceptable()) {
        ServerSocketChannel ssc = (ServerSocketChannel) key.channel();
        SocketChannel sc = ssc.accept();
        if (sc != null) {
            sc.configureBlocking(false);
            NioClientSession session = new NioClientSession(sc, this);
            sc.register(selector, SelectionKey.OP_READ, session);
            System.out.println("New connection from " + sc.getRemoteAddress());
        }
    } else if (key.isReadable()) {
        NioClientSession session = (NioClientSession) key.attachment();
        session.handleRead(); // 该方法内会将业务逻辑提交到线程池
    } else if (key.isWritable()) {
        NioClientSession session = (NioClientSession) key.attachment();
        session.handleWrite();
        if (session.writeQueue.isEmpty()) {
            key.interestOps(SelectionKey.OP_READ);
        }
    }
}

```

The start() method initializes the Selector, binds the ServerSocketChannel to port 9000 in non-blocking mode and registers the OP_ACCEPT interest; the main loop executes selector.select() and iterates over the selected keys. The 'handleKey()' method dispatches logic based on ACCEPT, READ and WRITE events: it creates an NioClientSession when accepting new connections, reads incoming data, and flushes the write queue.

Read/Write of NIO Session & Submission to Business Thread Pool

```

public void handleRead() throws IOException {
    int bytesRead = channel.read(readBuffer);
    if (bytesRead == -1) {
        disconnect();
        return;
    }

    readBuffer.flip();
    MessageCodec codec = new MessageCodec();
    String message;
    while ((message = codec.decode(readBuffer)) != null) {
        // 将消息处理提交到业务线程池, 避免阻塞 Selector 线程
        final String msg = message;
        server.getBusinessExecutor().submit(() -> {
            server.getRouter().route(this, this.getUsername(), msg);
        });
    }
    readBuffer.compact();
}

public void handleWrite() throws IOException {
    ByteBuffer buf;
    while ((buf = writeQueue.peek()) != null) {
        channel.write(buf);
        if (buf.hasRemaining()) {
            return;
        }
        writeQueue.poll();
    }
}

public void enqueueWrite(String message) {
    byte[] bytes = (message + "\n").getBytes(java.nio.charset.StandardCharsets.UTF_8);
    ByteBuffer buf = ByteBuffer.wrap(bytes);
    writeQueue.offer(buf);
    SelectionKey key = channel.keyFor(server.getSelector());
    if (key != null) {
        key.interestOps(SelectionKey.OP_READ | SelectionKey.OP_WRITE);
        server.getSelector().wakeup();
    }
}
}

```

handleRead(): invoke channel.read(), decode data via MessageCodec.decode(), then submit the decoded message to the business thread pool by calling server.getBusinessExecutor().submit();

handleWrite(): fetch ByteBuffer instances from ConcurrentLinkedQueue and write them to the channel;

enqueueWrite(): append the newline character \n, wrap the data into a ByteBuffer and enqueue it, register the OP_WRITE interest and wake up the selector.

MessageCodec: decode messages split by line breaks

```

public String decode(ByteBuffer buffer) {
    int start = buffer.position();
    int limit = buffer.limit();
    for (int i = start; i < limit; i++) {
        if (buffer.get(i) == (byte) '\n') {
            int length = i - start;
            if (length > 0 && buffer.get(i - 1) == (byte) '\r') {
                length--;
            }
            byte[] lineBytes = new byte[length];
            buffer.position(start);
            buffer.get(lineBytes, 0, length);
            buffer.position(i + 1);
            return new String(lineBytes, StandardCharsets.UTF_8);
        }
    }
    buffer.position(start);
    return null;
}

```

Iterate over the ByteBuffer to search for `\n` (compatible with `\r\n`), extract complete lines, decode them with UTF-8 and return the result. If no line terminator is found, reset the buffer position and return null (waiting for incomplete packet data).

二、 User Authentication & SHA-256 with Salt Password Hashing

Clients send protocol messages in the format:

LOGIN|username||password or REGISTER|username||password.

The server-side MessageRouter invokes UserManager for processing. Password storage adopts the SHA-256 plus random salt scheme: generate a 16-byte salt via SecureRandom, compute SHA-256(salt + password), then store the pair in users.dat as salt:hash. During verification, extract the stored salt, recalculate the hash, and perform constant-time comparison to mitigate timing attacks.

SHA-256 Hashing with Salt and Constant-Time Verification

```

public static String hash(String plainPassword) {
    if (plainPassword == null || plainPassword.isEmpty()) {
        throw new IllegalArgumentException("密码不能为空");
    }

    // 1. 生成随机盐
    String salt = generateSalt();

    // 2. 计算 salt + password 的 SHA-256
    String hash = sha256(salt + plainPassword);

    // 3. 返回 salt:hash
    return salt + ":" + hash;
}

/**
 * 验证明文密码是否与 {@code salt:hash} 存储值匹配。
 *
 * @param plainPassword 明文密码
 * @param storedValue 之前通过 {@link #hash(String)} 生成的存储值
 * @return true 表示密码匹配
 */
public static boolean verify(String plainPassword, String storedValue) {
    if (plainPassword == null || storedValue == null) {
        return false;
    }

    // 1. 解析 salt:hash
    int colonIndex = storedValue.indexOf(':');
    if (colonIndex <= 0 || colonIndex == storedValue.length() - 1) {
        return false; // 格式不合法
    }

    String salt = storedValue.substring(0, colonIndex);
    String expectedHash = storedValue.substring(colonIndex + 1);

    // 2. 用相同的盐重新计算哈希
    String computedHash = sha256(salt + plainPassword);

    // 3. 常量时间比对 (防止时序攻击)
    return constantTimeEquals(expectedHash, computedHash);
}

```

hash(): generate a random salt, compute the SHA-256 digest, and return the string formatted as salt:hash;

verify(): parse the stored salt value, recalculate the hash, and invoke constantTimeEquals() to perform bitwise XOR comparison for consistent execution duration.

User Registration & Login + Data Persistence

```

public AuthResult register(String username, String password) {
    if (username == null || password == null || username.isEmpty() || password.isEmpty()) {
        return AuthResult.fail("用户名或密码不能为空");
    }
    if (users.containsKey(username)) {
        return AuthResult.fail("用户名已存在");
    }
    String salt = generateSalt();
    String hash = hashPassword(password, salt);
    users.put(username, new User(username, salt, hash));
    saveUsers();
    return AuthResult.success();
}

// ----- 登录 -----
public AuthResult login(String username, String password) {
    if (username == null || password == null) {
        return AuthResult.fail("用户名或密码不能为空");
    }
    User user = users.get(username);
    if (user == null) {
        return AuthResult.fail("用户名不存在");
    }
    String computedHash = hashPassword(password, user.salt);
    if (!computedHash.equals(user.passwordHash)) {
        return AuthResult.fail("密码错误");
    }
    onlineUsers.add(username);
    return AuthResult.success();
}

// ----- 设置离线 -----
public void setUserOffline(String username) {
    if (username != null) {
        onlineUsers.remove(username);
    }
}

// ----- 纯验证（不改变在线状态） -----
public boolean authenticate(String username, String password) {
    if (username == null || password == null) return false;
    User user = users.get(username);
    if (user == null) return false;
    return hashPassword(password, user.salt).equals(user.passwordHash);
}

private String hashPassword(String password, String salt) {
    try {
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        md.update(salt.getBytes());
        byte[] hash = md.digest(password.getBytes());
        StringBuilder sb = new StringBuilder();
        for (byte b : hash) sb.append(String.format("%02x", b));
        return sb.toString();
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}

```

register(): Check username uniqueness, generate credential, store in ConcurrentHashMap, then persist data to users.dat via saveUsers();

login(): Recalculate hash with stored salt for comparison;

authenticate(): Perform authentication only without modifying online status;

hashPassword(): Implemented by MessageDigest.getInstance("SHA-256").

Authentication Message Routing

```

private void handleLogin(NioClientSession session, String username, String password) {
    if (OnlineUserManager.isBanned(username)) {
        System.out.println("You are banned.");
        return;
    }
    boolean ok = SecurityBootstrap.authenticate(username, password);
    if (ok) {
        NioClientSession old = OnlineUserManager.getSession(username);
        if (old != null) {
            old.enqueueWrite("SYSTEM|||您已在别处登录, 将被踢下线");
            old.disconnect();
        }
        session.setUsername(username);
        OnlineUserManager.addUser(username, session);
        session.enqueueWrite("SUCCESS|SYSTEM||登录成功");
        System.out.println("User " + username + " logged in.");
    } else {
        session.enqueueWrite("FAIL|SYSTEM||用户名或密码错误");
    }
}

// ===== 注册 =====
private void handleRegister(NioClientSession session, String username, String password) {
    UserManager um = UserManager.getInstance();
    AuthResult result = um.register(username, password);
    if (result.isSuccess()) {
        session.enqueueWrite("SUCCESS|SYSTEM||注册成功");
        System.out.println("User " + username + " registered.");
    } else {
        session.enqueueWrite("FAIL|SYSTEM||" + result.getMessage());
    }
}
}

```

handleLogin(): Check ban status, call SecurityBootstrap.authenticate(), kick out the old session, invoke OnlineUserManager.addUser(), and reply with SUCCESS;

handleRegister(): Call UserManager.register().

三、End-to-End Encryption with SSL/TLS

Load certificates from the JKS keystore via SSLContextBuilder, create an SSLContext, and generate an SSLServerSocketFactory. Toggle between plaintext and encrypted modes through the useSSL switch; TLSv1.2 is used by default.

Note: The current NIO implementation relies on ServerSocketChannel. SSL encryption and decryption are implemented at the channel layer using SSLEngine. The SSLContextBuilder provides a fully implemented certificate management infrastructure.

SSL Context Initialization (JKS Keystore → SSLContext)


```

public static SSLContext initSSL(String keystorePath,
                                String keystorePassword,
                                String keyPassword) throws Exception {
    // 1. 加载密钥库
    KeyStore keyStore = KeyStore.getInstance(KEYSTORE_TYPE);
    try (FileInputStream fis = new FileInputStream(keystorePath)) {
        keyStore.load(fis, keystorePassword.toCharArray());
    }

    // 2. 初始化 KeyManagerFactory
    KeyManagerFactory kmf = KeyManagerFactory.getInstance(
        KeyManagerFactory.getDefaultAlgorithm());
    kmf.init(keyStore, keyPassword.toCharArray());

    // 3. 初始化 TrustManagerFactory (信任同一密钥库中的证书)
    TrustManagerFactory tmf = TrustManagerFactory.getInstance(
        TrustManagerFactory.getDefaultAlgorithm());
    tmf.init(keyStore);

    // 4. 创建并初始化 SSLContext
    SSLContext context = SSLContext.getInstance(TLS_PROTOCOL);
    context.init(kmf.getKeyManagers(), tmf.getTrustManagers(), null);

    // 5. 缓存
    cachedContext = context;
    useSSL = true;

    System.out.println("[SSLContextBuilder] SSL 上下文初始化成功: " + keystorePath);
    return context;
}

```

Load keystore via `KeyStore.getInstance("JKS")`; use `KeyManagerFactory` to manage private key certificates and `TrustManagerFactory` to manage trusted certificates; initialize and cache `SSLContext.getInstance("TLSv1.2")`.

Factory Method for SSLServerSocket

```

public static ServerSocket getServerSocket(int port) throws IOException {
    if (useSSL) {
        SSLServerSocketFactory factory = getServerSocketFactory();
        if (factory == null) {
            throw new IOException("SSL 已启用但 SSLContext 未初始化, 请先调用 initSSL()");
        }
        return factory.createServerSocket(port);
    }
    return new ServerSocket(port);
}

```

`getServerSocket(port)`: Returns an `SSLServerSocket` or a standard `ServerSocket` depending on the `useSSL` flag.

四、UDP Broadcast Messages

A standalone `UDPBroadcastServer` thread listens on UDP port 9999. Upon receiving a UDP datagram, it invokes `ChatServer.broadcast()` to forward the message to all online TCP clients, enabling cross-protocol announcement broadcasting.

UDP Reception & Forwarding to TCP Broadcast

```

public void run() {
    try {
        socket = new DatagramSocket(UDP_PORT);
        Logger.getInstance().info("UDP Broadcast Server started on port " + UDP_PORT);
        byte[] buffer = new byte[4096];

        while (running) {
            DatagramPacket packet = new DatagramPacket(buffer, buffer.length);
            socket.receive(packet);
            String message = new String(packet.getData(), 0, packet.getLength());
            Logger.getInstance().debug("UDP received: " + message);

            // 将收到的消息通过 TCP 广播给所有在线用户
            if (chatServer != null) {
                chatServer.broadcast("[UDP广播] " + message);
            } else {
                Logger.getInstance().warning("ChatServer reference is null, cannot broadcast.");
            }
        }
    } catch (SocketException e) {
        if (!running) {
            Logger.getInstance().info("UDP socket closed normally.");
        } else {
            Logger.getInstance().error("UDP Socket error: " + e.getMessage());
        }
    } catch (IOException e) {
        Logger.getInstance().error("UDP receive error: " + e.getMessage());
    } finally {
        close();
    }
}
}

```

Initialize DatagramSocket bound to port 9999; block on socket.receive(packet) to receive datagrams, decode the payload into a string, then call chatServer.broadcast("[UDP Broadcast] " + message) to forward the content to all online TCP clients. This thread is started at lines 35–37 in ExtensionManager.java.

五、HTTP File Sharing and File Transfer

A dual-channel design is adopted:

HTTP Channel: An HttpServer listens on port 8080, exposing POST /upload and GET /download endpoints. Clients communicate via HttpURLConnection, and uploaded files are stored under the server_files/ directory.

TCP Socket Channel: Uses FILE_UPLOAD protocol messages. Files are encoded in Base64 and transmitted over Socket connections; the server decodes and persists the files, then broadcasts the downloadable link to all clients.

HTTP File Server Startup & Upload/Download Handler

```

public void start() throws IOException {
    // 创建存储目录
    File storage = new File(STORAGE_DIR);
    if (!storage.exists()) {
        storage.mkdirs();
    }

    server = HttpServer.create(new InetSocketAddress(PORT), 0);
    server.createContext("/upload", new UploadHandler());
    server.createContext("/download", new DownloadHandler());
    server.setExecutor(Executors.newCachedThreadPool());
    server.start();
    Logger.getInstance().info("File HTTP Server started on port " + PORT);
}
}

```

```

static class UploadHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange exchange) throws IOException {
        if (!"POST".equalsIgnoreCase(exchange.getRequestMethod())) {
            exchange.sendResponseHeaders(405, -1); // Method Not Allowed
            return;
        }

        // 从 URL 参数获取文件名, 例如 /upload?filename=myfile.txt
        String query = exchange.getRequestURI().getQuery();
        String filename = "uploaded_" + System.currentTimeMillis();
        if (query != null && query.startsWith("filename=")) {
            filename = URLDecoder.decode(query.substring(9), StandardCharsets.UTF_8.name());
        }
        filename = new File(filename).getName();

        // 保存文件
        Path filePath = Paths.get(STORAGE_DIR, filename);
        try (InputStream is = exchange.getRequestBody();
            FileOutputStream fos = new FileOutputStream(filePath.toFile())) {
            byte[] buffer = new byte[4096];
            int bytesRead;
            while ((bytesRead = is.read(buffer)) != -1) {
                fos.write(buffer, 0, bytesRead);
            }
        }

        String response = "File uploaded successfully: " + filename;
        exchange.sendResponseHeaders(200, response.getBytes().length);
        try (OutputStream os = exchange.getResponseBody()) {
            os.write(response.getBytes());
        }
        Logger.getInstance().info("File uploaded: " + filename);
    }
}

static class DownloadHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange exchange) throws IOException {
        if (!"GET".equalsIgnoreCase(exchange.getRequestMethod())) {
            exchange.sendResponseHeaders(405, -1);
            return;
        }

        String path = exchange.getRequestURI().getPath();
        String filename = URLDecoder.decode(path.substring(path.lastIndexOf('/') + 1), StandardCharsets.UTF_8.name());
        filename = new File(filename).getName();
        if (filename.isEmpty()) {
            exchange.sendResponseHeaders(400, -1); // Bad Request
            return;
        }

        Path filePath = Paths.get(STORAGE_DIR, filename);
        File file = filePath.toFile();
        if (!file.exists()) {
            exchange.sendResponseHeaders(404, -1); // Not Found
            return;
        }

        exchange.getResponseHeaders().add("Content-Type", "application/octet-stream");
        exchange.getResponseHeaders().add("Content-Disposition", "attachment; filename=\"" + filename + "\"");
        exchange.sendResponseHeaders(200, file.length());
        try (OutputStream os = exchange.getResponseBody();
            FileInputStream fis = new FileInputStream(file)) {
            byte[] buffer = new byte[4096];
            int bytesRead;
            while ((bytesRead = fis.read(buffer)) != -1) {
                os.write(buffer, 0, bytesRead);
            }
        }
        Logger.getInstance().info("File downloaded: " + filename);
    }
}

```

HttpServer.create(8080) → register /upload and /download contexts with a cached thread pool;

UploadHandler: Validate POST requests, extract filename from URL query

parameter ?filename=xxx, write file stream into the server_files/ directory;

DownloadHandler: Validate GET requests, set response header Content-Disposition: attachment, output file via stream.

HTTP Client File Upload

```
public boolean uploadFile(String filePath, String serverBaseUrl, String remoteFileName) {
    try {
        File file = new File(filePath);
        if (!file.exists() || !file.isFile()) {
            Logger.getInstance().error("File not found: " + filePath);
            return false;
        }

        String urlStr = serverBaseUrl + "/upload?filename=" + URLEncoder.encode(remoteFileName, "UTF-8");
        URL url = new URL(urlStr);
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("POST");
        conn.setDoOutput(true);
        conn.setConnectTimeout(5000);
        conn.setReadTimeout(5000);

        try (FileInputStream fis = new FileInputStream(file);
             OutputStream os = conn.getOutputStream()) {
            byte[] buffer = new byte[4096];
            int bytesRead;
            while ((bytesRead = fis.read(buffer)) != -1) {
                os.write(buffer, 0, bytesRead);
            }
        }

        int responseCode = conn.getResponseCode();
        if (responseCode == 200) {
            Logger.getInstance().info("Upload successful: " + remoteFileName);
            return true;
        } else {
            Logger.getInstance().error("Upload failed, HTTP code: " + responseCode);
            return false;
        }
    } catch (IOException e) {
        Logger.getInstance().error("Upload error: " + e.getMessage());
        return false;
    }
}
```

Construct the URL as `"/upload?filename=" + URLEncoder.encode(...)`, invoke `url.openConnection()` to obtain an `HttpURLConnection`, configure the connection as POST and enable `doOutput`, read local file content via `FileInputStream`, and write data to the request body through `getOutputStream()`.

TCP Socket File Upload — Client Transmission & Server Reception

```

private void sendFile() {
    JFileChooser chooser = new JFileChooser();
    if (chooser.showOpenDialog(this) != JFileChooser.APPROVE_OPTION) {
        return;
    }
    File file = chooser.getSelectedFile();
    if (file == null || file.length() > 50 * 1024 * 1024) {
        JOptionPane.showMessageDialog(this, "文件过大 (最大50MB)");
        return;
    }

    // 默认广播给所有人, 不再弹窗选择
    final String finalTarget = "all";
    final File finalFile = file;

    setSendEnabled(false);
    fileBtn.setEnabled(false);

    new Thread(() -> {
        try {
            FileClient fileClient = new FileClient();
            boolean uploaded = fileClient.uploadFile(
                finalFile.getAbsolutePath(),
                ChatClient.getFileServerBaseUrl(),
                finalFile.getName()
            );
            if (!uploaded) {
                throw new IllegalStateException("HTTP 文件上传失败");
            }

            String message = "FILE|" + username + "|" + (finalTarget.isEmpty() ? "all" : finalTarget) + "|"
                + finalFile.getName();
            connection.send(message);

            SwingUtilities.invokeLater(() -> {
                appendMessage("系统", "文件 " + finalFile.getName() + " 上传成功, 已通知其他用户下载");
                setSendEnabled(true);
                fileBtn.setEnabled(true);
            });
        } catch (Exception e) {
            e.printStackTrace();
            SwingUtilities.invokeLater(() -> {
                JOptionPane.showMessageDialog(ChatWindow.this,
                    "文件上传失败: " + e.getMessage());
            });
        }
    });
}

```

```

private void handleFileUpload(NioClientSession session, String sender, String receiver, String data) {
    // ... 保持不变 ...
    try {
        int colonPos = data.indexOf(':');
        if (colonPos == -1) {
            session.enqueueWrite("ERROR|SYSTEM||无效的文件上传格式");
            return;
        }
        String encodedFilename = data.substring(0, colonPos);
        String base64 = data.substring(colonPos + 1);

        String filename = URLDecoder.decode(encodedFilename, StandardCharsets.UTF_8.name());
        base64 = base64.replaceAll("\\s", "");

        byte[] fileBytes;
        try {
            fileBytes = Base64.getDecoder().decode(base64);
        } catch (IllegalArgumentException e) {
            session.enqueueWrite("ERROR|SYSTEM||Base64解码失败: " + e.getMessage());
            return;
        }

        Path fileDir = Paths.get("server_files");
        if (!Files.exists(fileDir)) Files.createDirectories(fileDir);
        String safeFilename = new File(filename).getName();
        Path filePath = fileDir.resolve(safeFilename);
        Files.write(filePath, fileBytes);

        session.enqueueWrite("SUCCESS|SYSTEM||文件上传成功: " + safeFilename);

        String encodedUrlFilename = URLEncoder.encode(safeFilename, StandardCharsets.UTF_8.name());
        String downloadUrl = "http://127.0.0.1:8080/files/" + encodedUrlFilename;
        String broadcastMsg = "BROADCAST|SYSTEM|ALL|用户 " + sender + " 上传了文件: " + safeFilename +
            " (下载: " + downloadUrl + ")";
        OnlineUserManager.broadcast(broadcastMsg);

        System.out.println("文件上传: " + safeFilename + " 来自 " + sender);

    } catch (Exception e) {
        e.printStackTrace();
        session.enqueueWrite("ERROR|SYSTEM||文件上传失败: " + e.getMessage());
    }
}

```

Client-side `sendFile()`: Select a file via `JFileChooser`, verify the file size does not exceed 50 MB, encode the file content with Base64, then send the message formatted as `FILE_UPLOAD|username|all|filename:base64`.

Server-side `handleFileUpload()`: Parse the `filename:base64` payload, decode the Base64 data, write the file with `Files.write()`, and broadcast the download link to all users.

六、Client GUI and Message Sending & Receiving (Private Chat / Group Chat / Online User List)

Multi-window Swing architecture: `LoginFrame` leads to `ChatWindow` (group chat) and `PrivateChatWindow` (private chat). `ClientGUI.onMessage()` parses and dispatches messages following the `TYPE|sender|receiver|content` protocol format.

Group chat uses the `BROADCAST` message type; the server calls `groupBroadcast()` to forward messages to all users except the sender.

Private chat uses the `PRIVATE` message type; the server calls `sendToUser()` for point-to-point delivery.

The `/list` command queries all online users.

Main Chat Window UI Initialization & Group Message Sending

```
private void initUI() {
    setTitle("聊天室 - " + username);
    setSize(500, 400);
    setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
    setLocationRelativeTo(null);

    chatArea = new JTextArea();
    chatArea.setEditable(false);
    chatArea.setLineWrap(true);
    JScrollPane scrollPane = new JScrollPane(chatArea);
    scrollPane.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_ALWAYS);

    JPanel inputPanel = new JPanel(new BorderLayout());
    inputField = new JTextField();
    sendBtn = new JButton("发送");
    logoutBtn = new JButton("退出");
    fileBtn = new JButton("📎");

    JPanel btnPanel = new JPanel(new FlowLayout(FlowLayout.RIGHT));
    btnPanel.add(fileBtn);
    btnPanel.add(sendBtn);
    btnPanel.add(logoutBtn);

    inputPanel.add(inputField, BorderLayout.CENTER);
    inputPanel.add(btnPanel, BorderLayout.EAST);

    setLayout(new BorderLayout());
    add(scrollPane, BorderLayout.CENTER);
    add(inputPanel, BorderLayout.SOUTH);

    sendBtn.addActionListener(e -> sendMessage());
    logoutBtn.addActionListener(e -> logout());
    fileBtn.addActionListener(e -> sendFile());
    inputField.addActionListener(e -> sendMessage());

    addWindowListener(new java.awt.event.WindowAdapter() {
        @Override
        public void windowClosing(java.awt.event.WindowEvent e) {
            logout();
        }
    });
}

private void sendMessage() {
    String content = inputField.getText().trim();
    if (content.isEmpty()) return;

    setSendEnabled(false);
    inputField.setEnabled(false);

    new Thread(() -> {
        try {
            String message = Protocol.BROADCAST + "|" + username + "|all|" + content;
            connection.send(message);

            SwingUtilities.invokeLater(() -> {
                appendMessage(username, content);
            });

            SwingUtilities.invokeLater(() -> {
                inputField.setText("");
                setSendEnabled(true);
                inputField.setEnabled(true);
                inputField.requestFocus();
            });
        } catch (Exception e) {
            e.printStackTrace();
            SwingUtilities.invokeLater(() -> {
                JOptionPane.showMessageDialog(ChatWindow.this,
                    "发送失败: " + e.getMessage());
                setSendEnabled(true);
                inputField.setEnabled(true);
            });
        }
    }).start();
}
```

BorderLayout layout: read-only JTextArea at center, JTextField at bottom, plus Send, Exit and

attachment clip buttons.

sendMessage(): construct and send message in format BROADCAST|username|all|content, append message content to local display.

Private Chat Window Message Sending & Receiving

```
private void sendPrivateMessage() {
    String content = inputField.getText().trim();
    if (content.isEmpty()) return;

    try {
        Message msg = new Message(Protocol.PRIVATE, currentUser, targetUser, content);
        connection.send(msg.encode());
        appendOwnMessage(content);
        inputField.setText("");
    } catch (Exception e) {
        JOptionPane.showMessageDialog(this, "私聊发送失败: " + e.getMessage());
    }
}

public void appendIncomingMessage(String sender, String content) {
    SwingUtilities.invokeLater(() -> {
        chatArea.append("[ " + sender + " ] " + content + "\n");
        chatArea.setCaretPosition(chatArea.getDocument().getLength());
    });
}

public void appendOwnMessage(String content) {
    SwingUtilities.invokeLater(() -> {
        chatArea.append("[我] " + content + "\n");
        chatArea.setCaretPosition(chatArea.getDocument().getLength());
    });
}
```

sendPrivateMessage(): Construct and send message formatted as PRIVATE|currentUser|targetUser|content;

appendIncomingMessage() / appendOwnMessage(): Append messages to chat panel via SwingUtilities.

Message Dispatch Controller

```
if (Protocol.PRIVATE.equals(type)) {
    String currentUser = (currentWindow != null) ? currentWindow.getUsername() : null;
    String target = null;
    if (currentUser != null && currentUser.equals(sender)) {
        target = receiver;
    } else if (currentUser != null && currentUser.equals(receiver)) {
        target = sender;
    }
    if (target != null) {
        PrivateChatWindow pWindow = privateChats.get(target);
        if (pWindow == null) {
            pWindow = new PrivateChatWindow(
                ChatClient.getConnection(),
                currentUser,
                target
            );
            pWindow.setVisible(true);
        }
        pWindow.appendIncomingMessage(sender, content);
    }
    return;
}

// ---- 广播/群聊 ----
if (Protocol.BROADCAST.equals(type) || "GROUP".equals(type)) {
    if (currentWindow != null) {
        currentWindow.appendMessage(sender, content);
    }
    return;
}
```

Parse protocol messages:

PRIVATE type: create or locate the corresponding private chat window and append the message;
BROADCAST/GROUP type: append the message to the main chat window.

Server-side Private & Group Chat Routing

```
private void handlePrivate(NioClientSession session, String sender, String receiver, String content) {
    if (!OnlineUserManager.isOnline(receiver)) {
        if (session != null) {
            session.enqueueWrite("ERROR|SYSTEM||User " + receiver + " is offline");
        }
        return;
    }
    String msg = "PRIVATE|" + sender + "|" + receiver + "|" + content;
    OnlineUserManager.sendToUser(receiver, msg);
}

// ===== 群聊 / 命令 =====
private void handleGroup(NioClientSession session, String sender, String content) {
    if (content.startsWith("/")) {
        handleAdminCommand(session, sender, content);
        return;
    }
    String msg = "BROADCAST|" + sender + "|ALL|" + content;
    OnlineUserManager.groupBroadcast(sender, msg);
}

public static void sendToUser(String username, String message) {
    NioClientSession session = onlineUsers.get(username);
    if (session != null) {
        session.enqueueWrite(message);
    }
}

public static String getOnlineList() {
    return String.join(",", onlineUsers.keySet());
}

public static void groupBroadcast(String username, String msg) {
    for (Map.Entry<String, NioClientSession> entry : onlineUsers.entrySet()) {
        if (!entry.getKey().equals(username)) {
            entry.getValue().enqueueWrite(msg);
        }
    }
}

public static void broadcast(String msg) {
    for (Map.Entry<String, NioClientSession> entry : onlineUsers.entrySet()) {
        entry.getValue().enqueueWrite(msg);
    }
}
```

handlePrivate(): Check if target user is online, then deliver message point-to-point via sendToUser();

handleGroup(): Invoke groupBroadcast() to broadcast to all users except the message sender.

In OnlineUserManager:

sendToUser(): enqueue messages asynchronously in non-blocking mode by calling enqueueWrite();

groupBroadcast(): iterate over all online users and skip the sender;

broadcast(): send message to every online user.

Online User List

```

private String listUsers() {
    Set<String> users = chatServer.getOnlineUsers();
    if (users.isEmpty()) return "当前没有在线用户。";
    return "在线用户: " + String.join(", ", users);
}

public static Set<String> getUsersList() {
    return onlineUsers.keySet();
}

```

listUsers(): Call getOnlineUsers() to retrieve the key set of the ConcurrentHashMap, concatenate keys with commas and return the result.

七、Server-Side User Session Management & Administrator Commands

OnlineUserManager maintains session mappings via ConcurrentHashMap<String, NioClientSession>, handling login, registration and disconnection cleanup. Multi-login detection kicks out the old session when a new connection from the same user arrives.

AdminCommandHandler intercepts commands prefixed with slash /. Users with username admin or root are granted administrator privileges.

Commands /help, /tell and /download require no permissions; /kick, /ban, /unban, /list, /broadcast and /shutdown are restricted to administrators only.

Session Management + User Kicking / Banning

```

private static final Map<String, NioClientSession> onlineUsers = new ConcurrentHashMap<>();
private static final Set<String> bannedList = new HashSet<>();

public static void addUser(String username, NioClientSession session) {
    onlineUsers.put(username, session);
}

public static void removeUser(String username) {
    onlineUsers.remove(username);
}

public static NioClientSession getSession(String username) {
    return onlineUsers.get(username);
}

public static boolean isOnline(String username) {
    return onlineUsers.containsKey(username);
}

public static void sendToUser(String username, String message) {
    NioClientSession session = onlineUsers.get(username);
    if (session != null) {
        session.enqueueWrite(message);
    }
}

public static String getOnlineList() {
    return String.join(", ", onlineUsers.keySet());
}

public static void groupBroadcast(String username, String msg) {
    for (Map.Entry<String, NioClientSession> entry : onlineUsers.entrySet()) {
        if (!entry.getKey().equals(username)) {
            entry.getValue().enqueueWrite(msg);
        }
    }
}

public static void broadcast(String msg) {
    for (Map.Entry<String, NioClientSession> entry : onlineUsers.entrySet()) {
        entry.getValue().enqueueWrite(msg);
    }
}

```

```

public static boolean kick(String username) {
    NioClientSession session = onlineUsers.get(username);
    if (session != null) {
        onlineUsers.remove(username);
        session.disconnect(); // 关闭连接
        return true;
    }
    return false;
}

public static boolean isBanned(String username) {
    return bannedList.contains(username);
}

public static boolean ban(String username) {
    NioClientSession session = onlineUsers.get(username);
    if (session != null) {
        onlineUsers.remove(username);
        bannedList.add(username);
        session.disconnect(); // 关闭连接
        return true;
    }
    return false;
}

```

Username are mapped to NioSession instances and stored in a ConcurrentHashMap; exposed methods include addUser(), removeUser(), sendToUser(), groupBroadcast() and broadcast().

kick(): Remove the entry from the map and close the channel via disconnect();

ban(): Additionally add the target user to bannedList.

Admin Command Handler

```

public String handleCommand(String fullMessage, String sender) {
    if (fullMessage == null || !fullMessage.startsWith("/")) {
        return "不是有效命令。";
    }

    String[] parts = fullMessage.split("\\s+", 2);
    String command = parts[0].toLowerCase();
    String arg = parts.length > 1 ? parts[1] : "";

    // 1. 先检查命令是否已知
    if (!KNOWN_COMMANDS.contains(command)) {
        return "未知命令, 输入 /help 查看帮助。";
    }

    // 2. /help、/tell、/download 不需要管理员权限
    if ("/help".equals(command)) {
        return getHelp();
    }
    if ("/tell".equals(command)) {
        return tell(sender, arg);
    }
    if ("/download".equals(command)) {
        return download(sender, arg.trim());
    }

    // 3. 其他命令需要管理员权限
    if (!admins.contains(sender)) {
        return "权限不足, 只有管理员可以执行此命令。";
    }

    switch (command) {
        case "/kick":
            return kick(arg.trim());
        case "/ban":
            return ban(arg.trim());
        case "/unban":
            return unban(arg.trim());
        case "/list":
            return listUsers();
        case "/broadcast":
            return broadcastMessage(arg);
        case "/shutdown":
            return shutdownServer();
        default:
            return "未知命令, 输入 /help 查看帮助。";
    }
}

```

Split commands and arguments, validate supported command types. Commands /help, /tell and /download require no privileges; all other commands verify admin permissions before execution. Dispatch logic via switch-case for /kick, /ban, /unban, /list, /broadcast and /shutdown.

八、Server Activity Logging

A singleton Logger supporting four log levels: INFO, WARNING, ERROR and DEBUG. Dual output channels are provided: console output via System.out.println, and persistent file output to logs/app-yyyy-MM-dd.log. The file writer uses append mode with daily log rolling. Each log entry contains a millisecond-precision timestamp.

Full Logger Implementation

```

public class Logger {
    private static Logger instance;
    private final String logDir = "logs/";
    private final DateTimeFormatter dateFormatter = DateTimeFormatter.ofPattern("yyyy-MM-dd");
    private final DateTimeFormatter timeFormatter = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss.SSS");

    private Logger() {
        File dir = new File(logDir);
        if (!dir.exists()) {
            dir.mkdirs();
        }
    }

    public static synchronized Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

    private void log(String level, String message) {
        String timestamp = LocalDateTime.now().format(timeFormatter);
        String logLine = String.format("[%s] [%s] %s", timestamp, level, message);
        // 控制台输出
        System.out.println(logLine);
        // 写入文件
        String fileName = logDir + "app-" + LocalDateTime.now().format(dateFormatter) + ".log";
        try (FileWriter fw = new FileWriter(fileName, true);
            BufferedWriter bw = new BufferedWriter(fw);
            PrintWriter out = new PrintWriter(bw)) {
            out.println(logLine);
        } catch (IOException e) {
            System.err.println("Failed to write log: " + e.getMessage());
        }
    }

    public void info(String msg) {
        log("INFO", msg);
    }

    public void warning(String msg) {
        log("WARNING", msg);
    }

    public void error(String msg) {
        log("ERROR", msg);
    }

    public void debug(String msg) {
        log("DEBUG", msg);
    }
}

```

Singleton pattern → generate millisecond timestamp via `LocalDateTime.now()` → format log entry as `[timestamp] [level] message` → print to console and write to `logs/app-yyyy-MM-dd.log` daily by date → four public methods for INFO, WARNING, ERROR and DEBUG levels.

九、IPv6 Compatibility and DNS Resolution

Call `InetAddress.getAllByName(host)` to resolve all IPv4 and IPv6 addresses in one batch, iterate and attempt connections sequentially, return upon the first successful connection. Literal IPv6 addresses are wrapped in square brackets (e.g. `[:,1]`) within URLs to comply with standard formatting rules.

DNS Resolution + Dual-Stack IPv4/IPv6 Connection

```

private Socket connect(String host, int port) throws IOException {
    InetAddress[] addresses = InetAddress.getAllByName(host);
    IOException lastException = null;

    for (InetAddress address : addresses) {
        Socket candidate = new Socket();
        try {
            candidate.connect(new InetSocketAddress(address, port), 5000);
            return candidate;
        } catch (IOException e) {
            lastException = e;
            try {
                candidate.close();
            } catch (IOException ignored) {}
        }
    }

    if (lastException != null) {
        throw lastException;
    }
    throw new UnknownHostException("无法解析主机: " + host);
}

```

Call `InetAddress.getAllByName(host)` to retrieve all IP addresses, iterate through each entry to create `InetSocketAddress(address, port)` and attempt a connection with a 5-second timeout. Return the first successfully connected address; throw `UnknownHostException` if all attempts fail.

IPv6 URL Format Handling

```

// private static final String SERVER_HOST = "localhost"; // DNS解析测试
private static final String SERVER_HOST = "127.0.0.1"; // IPv4测试
// private static final String SERVER_HOST = "::1"; // IPv6测试

private static String formatHostForUrl(String host) {
    if (host.contains(":") && !(host.startsWith("[") && host.endsWith("]"))) {
        return "[" + host + "]";
    }
    return host;
}

```

`SERVER_HOST` supports optional values: `localhost` (DNS), `127.0.0.1` (IPv4), or `::1` (IPv6); `formatHostForUrl()`: Wrap IPv6 addresses with square brackets (e.g. `::1`) to produce valid URLs.

Experimental Results

1. Start the server:

```

[2026-07-01 10:57:49.539] [INFO] UDPBroadcastServer started on port 9999
[2026-07-01 10:57:49.540] [INFO] AdminCommandHandler initialized
[2026-07-01 10:57:49.541] [INFO] ExtensionManager initialization completed.
ChatServer started on port 9000
[2026-07-01 10:57:49.546] [INFO] UDP Broadcast Server started on port 9999

```

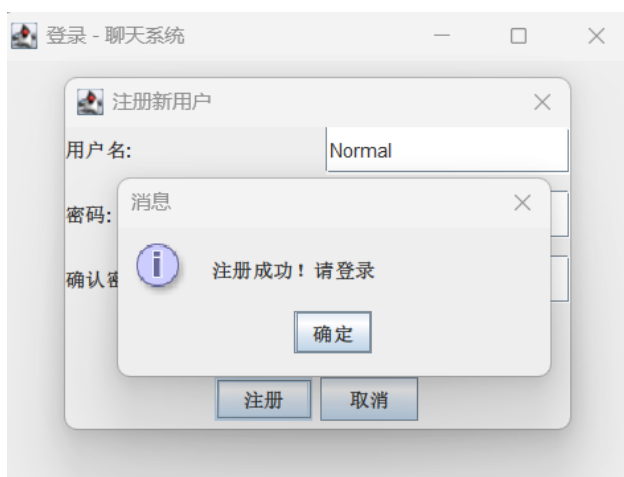
2. Start the clients (open the GUI automatically as follow):



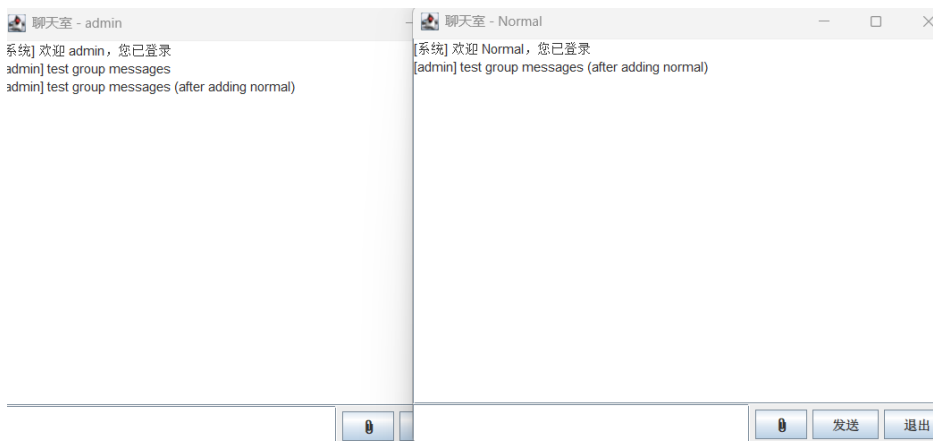
3. Login the administrator account:



4. Register a new normal account:

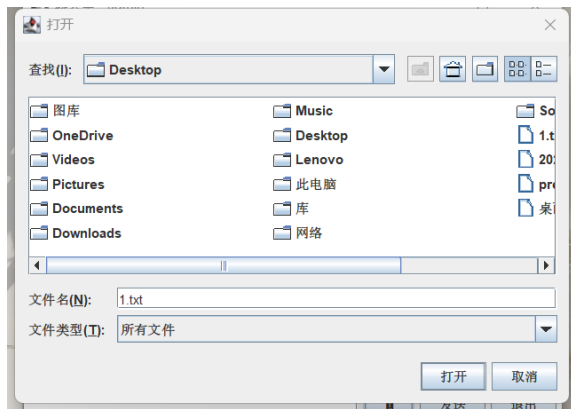


5. Test group messages sending:



6. Test file upload & download:

(uploaded files are stored in the /server_files folder)
(downloaded files are stored in the /downloads folder)

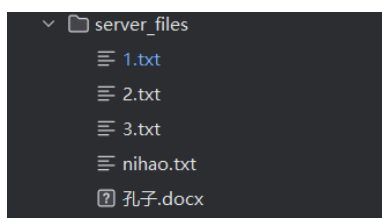


```
[系统] 欢迎 admin, 您已登录
admin] test group messages
admin] test group messages (after adding normal)
[系统] 文件 1.txt 上传成功, 已通知其他用户下载
```

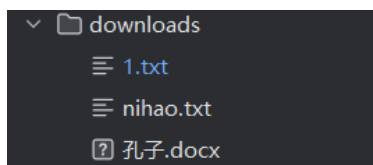


We can also use the /download command to download a specific file:
e.g. /download 2.txt

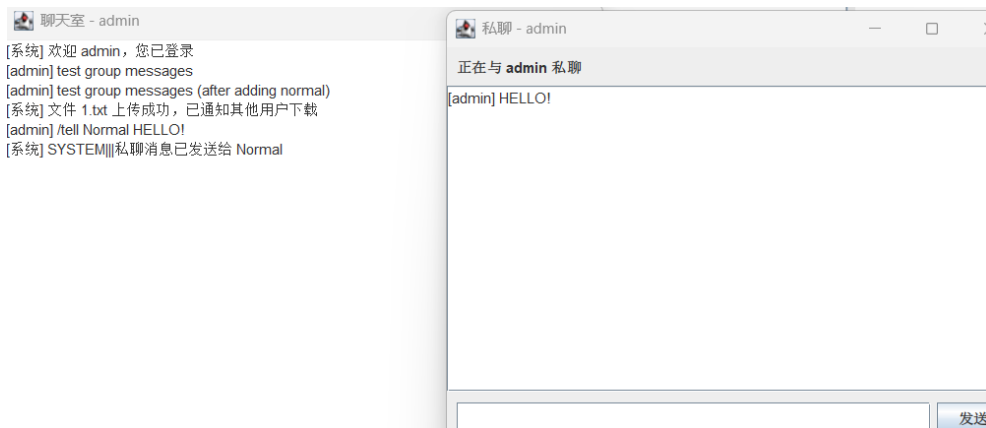
/server_files:



/downloads:

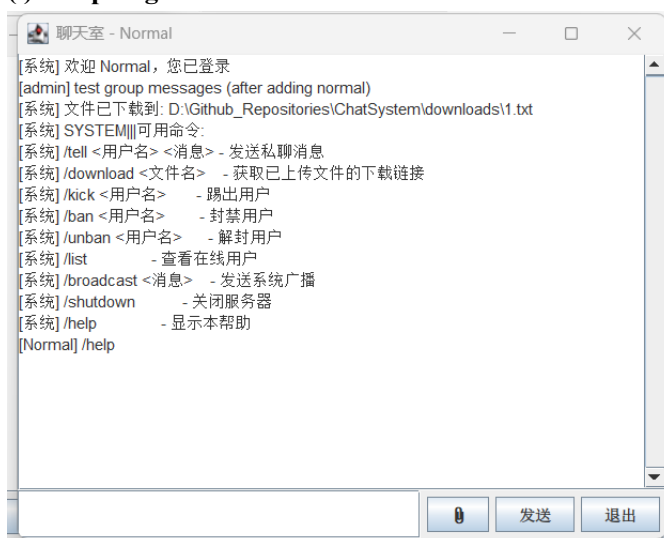


7. Test private messages sending:
(using command: /tell <username> <message>)



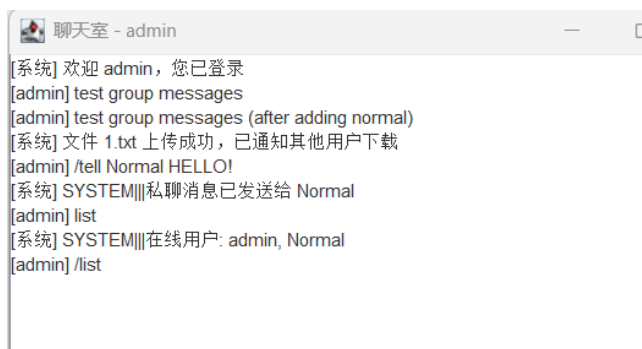
8. Test admin commands:

(i). /help to get the command list:



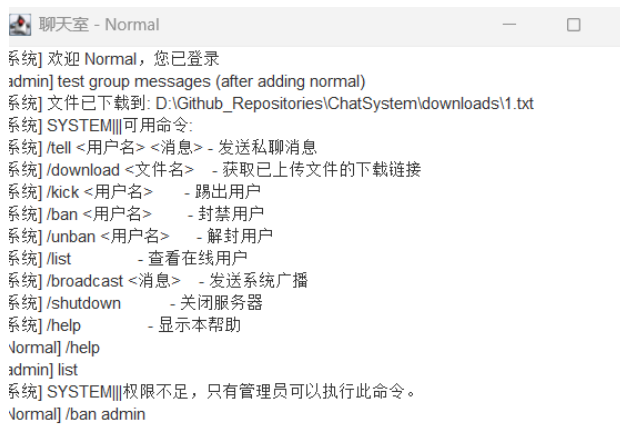
The following commands can only be executed in the administrator account:

(ii). /list to get the online user list:

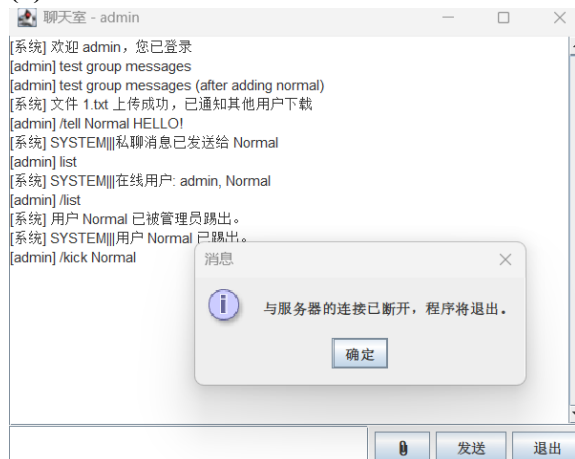


Other normal account who tried to use this command will cause a failure:

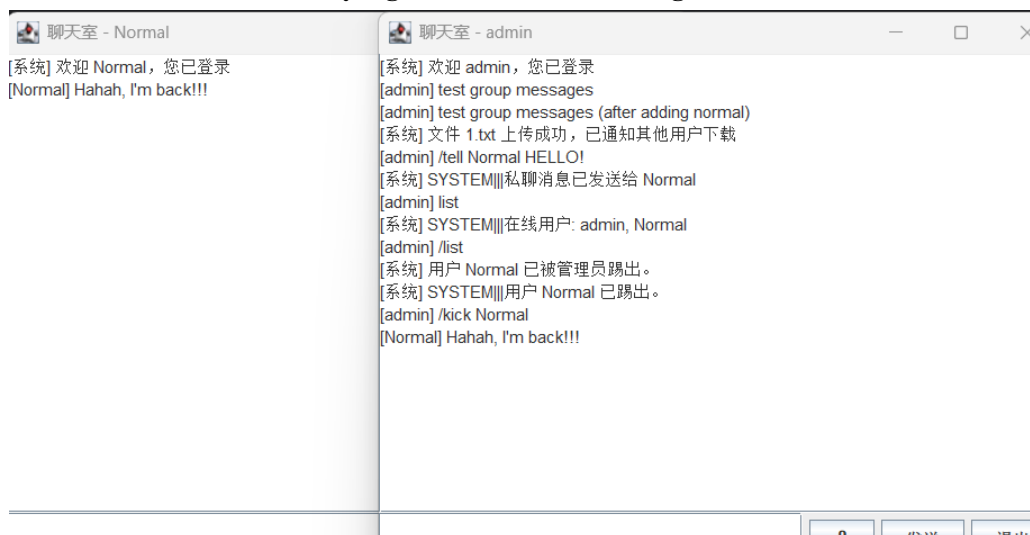
e.g. /ban



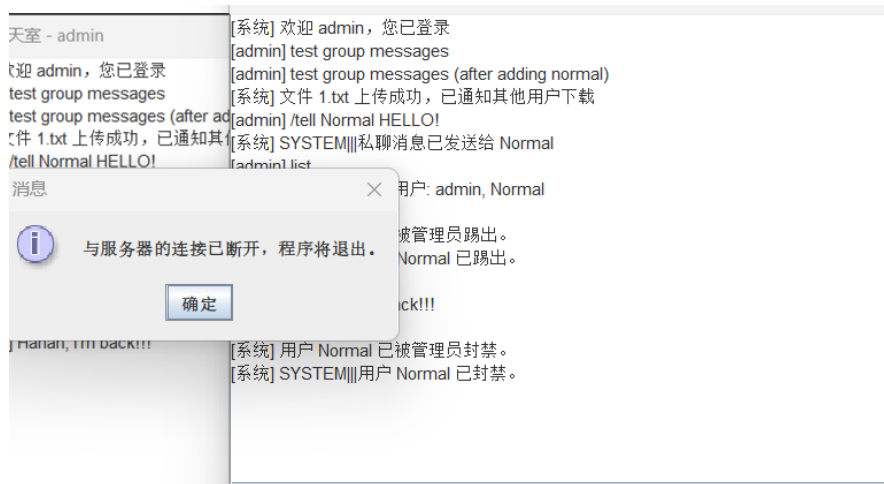
(ii). /kick to kick users:



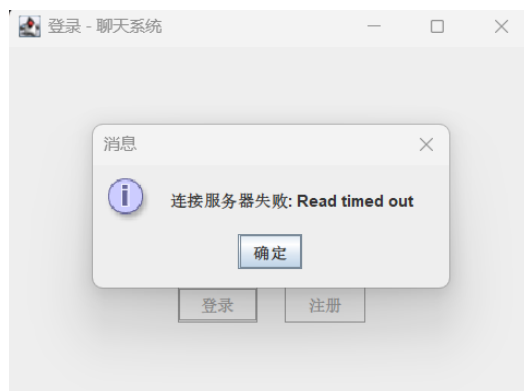
However, we can successfully login “Normal” account again:



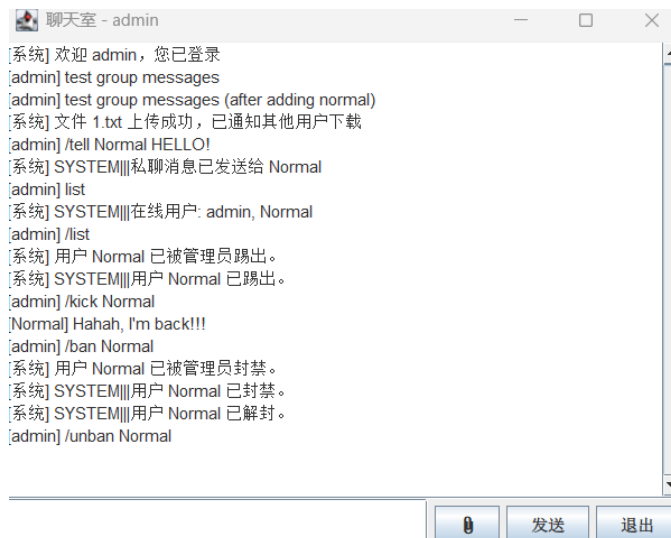
(iii). /ban : if the administrator didn't want to see this bad guy again, he/she can use the /ban command to ban the guy's account:



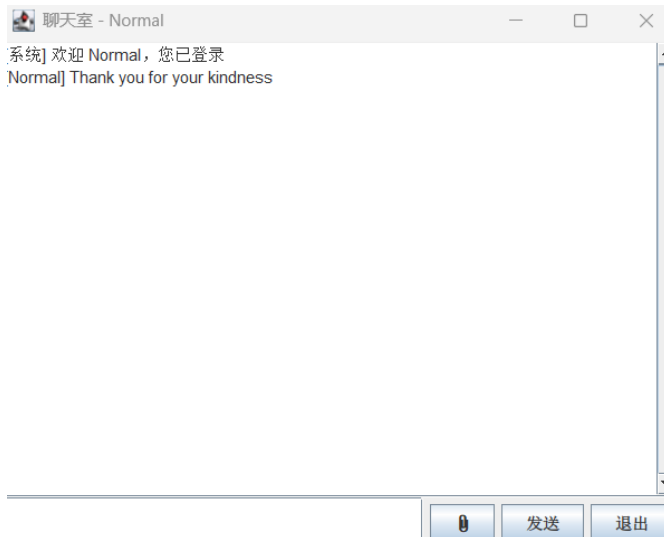
Notice that he can not login now:



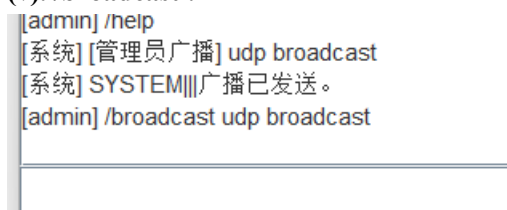
(iv). /unban : if the bad guy realized his fault and becomes a good person, we can use /unban command to unban his account:



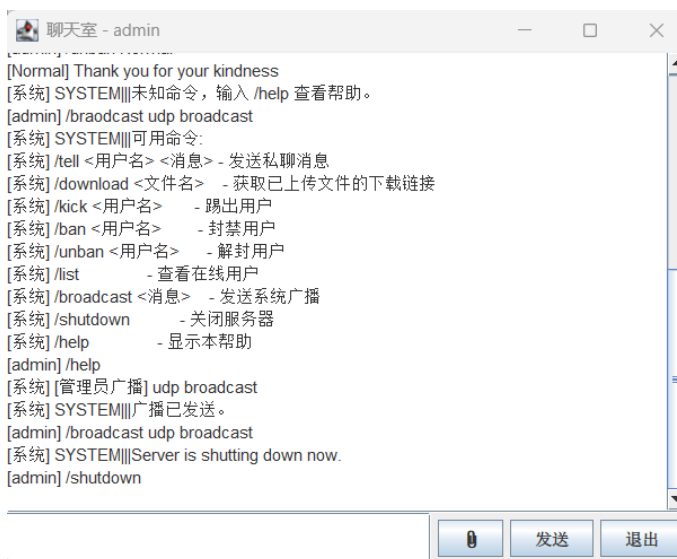
We can login again!



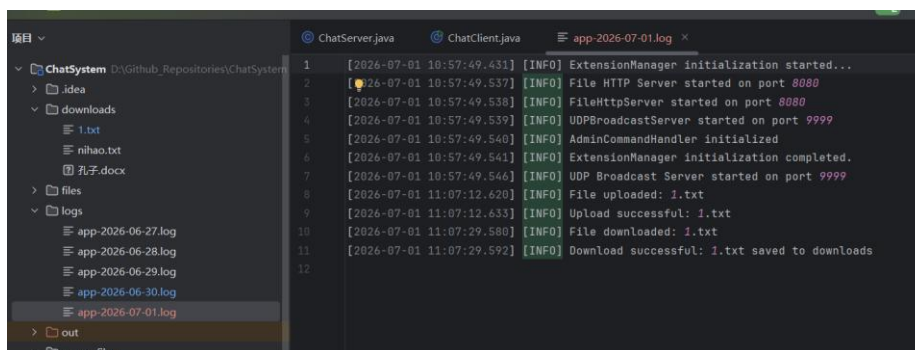
(v). /broadcast :



(vi). /shutdown:



9. Logs:



Detailed running logs are stored in the /logs folder.

10. users.dat:

```
ChatServer.java ChatClient.java app-2026-07-01.log users.dat x
1 TEST2:q06Co0s0Q0UDt81Hm6dqHg==:db2c6b90c1a6f49199044c689aed9289fdb56a89379aadb8223cef9050e481b
2 tt:oeBnwMDASb+VzW2i1lPr0g==:8f4d5e68e20dc76854669e82e477e9f5eb836a2d33e54d085e0770ecd53ec11f
3 CJG:1d7ngwCuaAQmqUv0RZTzcg==:f063d2bf4b123f34e2f0a6a92ed056c49c5798bdebbba1e21b6b04ca6351127d
4 instra:7QN3/rpDeXR6A0yI0qH59A==:e63eb3aaf9e2297c20455388dd55edd43bede3cfbeba7316fed0b4c0c82c34f4f
5 SZM:qs50PdyrdWs0ZCuPorNgSA==:20bc5071252cb298cc01a39118b364ec122df77d67bec3daf541cf443c5653d7
6 admin:B36cKNeKfQpCnf7i5jGjgQ==:dd23991c4a406fa54ee79efe1ed4e29eb870ee499d63b4a46ea684bc58c12e38
7 Normal:x1/p71yEz+2DQ1oLCywkRw==:524f0321aca820e18ddd44340f308bb69ad829988d83ed15b41bbb767d7af778
8 111:lhPKyw2yJrSyX0qHgIy/+A==:abb21bb9c1eef63792126c44d9686ac5cb515ff4aa444d5660c134cfc025fb5d
9 2z2:kWGZiJ1T1LQ8fNt5yJL2Lg==:859da0bd6b8cbb4946da82523e80666feb0015d5df1548b9ee14fb6e1fc4ce4
10 TEST:UyJa+BkChu0R5m3/Se5z2Q==:33dedccdebcb1059bc22ec6026ffbbab0b0122bba409e97f4efae6b5e625cd69a
11 root:z12Fm46R76WH0RGHY9S4g==:bbd22bdb427691346bb810fa300c6e03ca9bca59ab8df545d5272be83e06e72a
12 HZM:3vFABx8CDf0qsDFPqYgt/A==:3b4f8dc5ce5b62007532b17e61125312285463e152fbfe23c7517668aad34a50
13
```

Users' information is encrypted and stored in users.dat after registration.

Experimental Conclusions

This experiment implements a fully functional multi-user chat system built on the Java NIO and Swing technology stack. The server adopts a Selector multiplexing + thread pool architecture. A single thread efficiently manages massive concurrent connections, eliminating the resource waste of the traditional BIO model that allocates one independent thread for each connection.

The system covers core instant messaging capabilities: persistent TCP connection communication, group and private chat, user registration and login secured by SHA-256 with salt hashing, dual-channel HTTP file transfer, UDP broadcast announcements, administrator commands (user kicking, account banning, global broadcast), and a complete SSL/TLS encryption framework. The logging module records all runtime activities with daily rolling log files. The client supports dual-stack IPv4/IPv6 and DNS hostname resolution.

The overall design decouples I/O threads from business logic threads. Each module features clear responsibilities and excellent scalability, meeting practical requirements for concurrent multi-user chatting.

二、诚信承诺:

本人郑重承诺在该课程论文完成的过程中不发生任何不诚信现象，一切不诚信所导致的后果均由本人承担。

Integrity Commitment:

I/We solemnly promise that I/we have not plagiarized during the completion of this course paper, and all the consequences caused by plagiarism will be borne by me/us.

签名拍照附件:

Name and Digital Signature of all students in the group

苏泽铭 陈靖果
李培鑫 何展明

Date: July 1st , 2026